



# bonsai

Flutter Widget Complexity Analyzer

Project Scope · Architecture · 6-Day Build Plan · Codex Prompts

Muhammad Kaif Nathani · SE-23024 · NED University · LetsDoIT Solutions

# 1. What is bonsai

bonsai is a static analysis tool for Flutter that measures widget tree health at the source-code level and produces a single health score (0-100) per file. It works without running your app, without a build step, and without the Flutter SDK. It reads your .dart files, walks the AST, finds every reactive scope, counts the widgets inside it, measures how well your screens are decomposed, and tells you where your rebuild risk is — before your users feel it.

## THE ONE-LINER

*A static analyzer for Flutter that estimates rebuild cost of your widget tree before your users feel it.*

## THE THREE METRICS — THESE ARE THE WHOLE PRODUCT

- **Blast Score** — for every reactive scope (setState, StreamBuilder, Consumer, BlocBuilder, ValueListenableBuilder, FutureBuilder, AnimatedBuilder) count the number of widget constructor instantiations inside its subtree. The highest single-scope count in a file is its Blast Score. A Blast Score of 5 is fine. A Blast Score of 60 means 60 widgets rebuild on every state tick.
- **Composability Pressure** — ratio of raw leaf widgets (Text, Icon, SizedBox, Container, Padding, Divider) to extracted custom widget classes in a build() method. A build() with 40 Text/Icon leaves and 0 custom classes has pressure 0.0 — maximum decomposition needed. A build() with mostly custom widget classes has pressure 1.0 — well structured.
- **Context Coupling Depth** — the nesting level inside a build() method at which context.watch, context.read, context.select, or Provider.of is called. Depth 1 means controlled. Depth 9 means the entire tree above it rebuilds unnecessarily on every state change.

## HEALTH SCORE FORMULA — EXACT

```
health = 100 - clamp(blast_score / 2, 0, 40) # 40pt max penalty - clamp((1 - composability) * 30, 0, 30) # 30pt max penalty - clamp(coupling_depth * 3, 0, 30) # 30pt max penalty GREEN >= 75 well structured, no action needed YELLOW 50-74 review suggested before merging RED < 50 refactor required, CI gate fails
```

## WHAT DOES NOT EXIST TODAY — THE GAP YOU ARE FILLING

- dart analyze / flutter analyze — checks syntax and type errors. Knows nothing about widget tree structure or reactive scope.
- widget\_analyser (pub.dev) — reports nesting depth and cyclomatic complexity of build(). Blunt instrument. Depth-5 Column>Padding>Row>Text is perfectly fine and it will flag it anyway.
- DCM / Dart Code Metrics (commercial) — measures line count, parameter count, cognitive complexity. Has no concept of reactive scope. Cannot tell you the blast radius of a StreamBuilder.
- Flutter DevTools Widget Inspector — runtime only. You find the problem after users felt it. Cannot run in CI. Requires app to be running.

***bonsai is the first tool that statically models reactive scope blast radius. That sentence is what your README, your LinkedIn post, and your pub.dev description lead with.***

## 2. Architecture

---

### REPOSITORY STRUCTURE

```
bonsai/ packages/ bonsai_core/ pure Dart — AST parsing + metric engine lib/ bonsai_core.dart barrel
export src/ models.dart FileMetrics, HealthScore, ScopeInfo, AnalysisResult parser.dart parseFile()
+ analyzeFile() entry points visitor.dart BonsaiVisitor extends RecursiveAstVisitor scopes.dart
kReactiveScopes + kReactiveMethods constants subtree.dart countWidgetsInSubtree(AstNode) -> int
composability.dart kLeafWidgets + composability ratio logic context_detector.dart
ContextCallDetector visitor aggregator.dart HealthScore.fromMetrics() formula test/ fixtures/
sample dart files with known expected scores unit/ per-metric isolated tests integration/
analyzeFile() end-to-end tests pubspec.yaml bonsai_cli/ Dart CLI — dart pub global activate entry
bin/ bonsai.dart main() entry point lib/ src/ runner.dart AnalyzeCommand, glob file walking
formatter.dart pretty output + JSON output config.dart .bonsai.yaml loader pubspec.yaml
bonsai_vscode/ TypeScript VS Code extension src/ extension.ts activate(), gutter decorations, hover
provider cli_runner.ts shells out to bonsai CLI, parses JSON webview.ts workspace analysis panel
package.json tsconfig.json .github/ workflows/ ci.yml dart analyze + test on every push publish.yml
pub.dev publish on v*.*.* tag scripts/ benchmark.dart clones 10 OSS Flutter repos, runs bonsai,
prints table README.md .gitignore
```

### DATA FLOW — SOURCE FILE TO SCORE

```
.dart file path -> parser.dart: parseFile(path) using analyzer pkg -> CompilationUnit (AST) ->
visitor.dart: BonsaiVisitor walks AST -> scopes.dart: detect reactive scope nodes -> subtree.dart:
count widget constructors inside each scope -> composability.dart: count leaves vs custom classes ->
context_detector.dart: find context.watch/read depth -> models.dart: assemble FileMetrics ->
aggregator.dart: HealthScore.fromMetrics(fileMetrics) -> AnalysisResult { metrics, health } -> CLI:
pretty print or JSON -> VSCode: gutter annotation + hover tooltip
```

### KEY DEPENDENCY — THE ANALYZER PACKAGE

bonsai\_core depends on exactly one external package: analyzer: ^6.4.0 — the official Dart team AST package. It parses .dart files into a CompilationUnit tree without running the code and without needing the Flutter SDK. This is what makes bonsai\_core pure Dart and testable in isolation. The key classes you will use: ParsedUnitResult, CompilationUnit, MethodDeclaration, InstanceCreationExpression, MethodInvocation, RecursiveAstVisitor.

### 3. 6-Day Build Plan

Each day has a clear target and a clear done condition. Do not start the next day until the verify step passes. Days 1-2 are pure Dart. Day 3 is the hardest. Day 4 finishes the engine. Day 5 is CLI. Day 6 is VS Code extension + CI + launch.

**DAY 1**  
3-4 hrs

#### Scaffold + Models

Mono-repo structure, package stubs, all data models with no logic.

##### DELIVERABLES — ALL MUST BE TRUE BEFORE MOVING ON

- ✓ `bonsai_core` and `bonsai_cli` packages scaffolded with `dart create`
- ✓ `models.dart` complete with `FileMetrics`, `HealthScore`, `ScopeInfo`, `ContextCall`, `AnalysisResult`
- ✓ `HealthScore.fromMetrics()` formula implemented and verified with manual test values
- ✓ `dart pub get` passes in both packages with zero warnings
- ✓ `dart analyze` shows zero issues

##### VERIFY STEP

Run `HealthScore.fromMetrics` with `blast=60` — score must be below 50 (RED). Run with `blast=5`, `composability=0.8`, `depth=2` — score must be above 75 (GREEN).

**DAY 2**  
2-3 hrs

#### Constants + Parser Setup

Reactive scope constants, leaf widget list, parser wiring. No visitor logic yet.

##### DELIVERABLES — ALL MUST BE TRUE BEFORE MOVING ON

- ✓ `scopes.dart`: `kReactiveScopes` and `kReactiveMethods` lists defined
- ✓ `composability.dart`: `kLeafWidgets` list defined
- ✓ `parser.dart`: `parseFile(path)` implemented using analyzer `parseFile` API returning `CompilationUnit`
- ✓ `analyzeFile(path)` stub returns empty `FileMetrics` without crashing
- ✓ Parse error handling: broken `.dart` file returns `FileMetrics` with `parseError` flag, no exception thrown

##### VERIFY STEP

Create a test `.dart` file with a syntax error. Call `analyzeFile()` on it. It must return without throwing and have `parseError == true`.

**DAY 3**  
4-5 hrs

#### AST Visitor — Blast Score

Core of the entire tool. `BonsaiVisitor` walks AST, finds reactive scopes, counts widget subtrees.

##### DELIVERABLES — ALL MUST BE TRUE BEFORE MOVING ON

- ✓ `BonsaiVisitor` extends `RecursiveAstVisitor` implemented in `visitor.dart`
- ✓ `visitInstanceCreationExpression` detects all nodes in `kReactiveScopes` and records `ScopeInfo`
- ✓ `visitMethodInvocation` detects `setState` calls and records `ScopeInfo`
- ✓ `countWidgetsInSubtree(AstNode)` in `subtree.dart` recursively counts uppercase constructor instantiations
- ✓ Nesting depth counter increments on widget enter and decrements on widget exit
- ✓ `analyzeFile()` calls `BonsaiVisitor` and returns `FileMetrics` with `scopes` and `maxBlastScore` populated
- ✓ `heavy_screen.dart` fixture with `StreamBuilder` wrapping 50 widgets returns `maxBlastScore` near 50
- ✓ `clean_screen.dart` fixture with 3 custom widget classes returns `maxBlastScore` below 5

## VERIFY STEP

Print `FileMetrics` for both fixture files. Confirm `maxBlastScore` roughly equals widget count inside `StreamBuilder` for heavy fixture.

**DAY 4**  
3-4 hrs

## Composability + Context Depth + Tests

Remaining two metrics. Full unit test suite with known-input / known-output pairs.

### DELIVERABLES — ALL MUST BE TRUE BEFORE MOVING ON

- ✓ `composabilityRatio` computed from leaf vs custom widget counts in each `build()`
- ✓ `ContextCallDetector` finds `context.watch`, `context.read`, `context.select`, `Provider.of`
- ✓ `maxContextDepth` is max nesting level at which any context call occurs, reset per `build()` method
- ✓ All three metrics merged into `FileMetrics` via `analyzeFile()`
- ✓ `HealthScore.fromMetrics` tested with 5 known inputs covering GREEN YELLOW and RED bands
- ✓ Blast score unit tests with at least 6 inline AST cases
- ✓ Composability tests: all-leaves returns 0.0, all-custom returns 1.0, mixed returns correct ratio
- ✓ Context depth tests: no calls returns 0, depth-8 fixture returns 8
- ✓ Integration test: `analyzeFile` on all 4 fixtures asserts correct score band
- ✓ dart test passes with 0 failures, coverage  $\geq 90\%$  on `lib/src/`

## VERIFY STEP

Run `dart test --reporter=expanded`. Every group must show `PASSED`. Run `dart test --coverage` and confirm  $\geq 90\%$ .

**DAY 5**  
3-4 hrs

## CLI — bonsai analyze

User-facing CLI. Pretty colour output, JSON output, CI exit code flag.

### DELIVERABLES — ALL MUST BE TRUE BEFORE MOVING ON

- ✓ `bonsai analyze` globs all `.dart` files excluding `*.g.dart` and `*.freezed.dart`
- ✓ Pretty output: filename padded to 40 chars, score/100, ANSI colour band, suggestions indented below
- ✓ JSON output via `--format json`: pure JSON array of `AnalysisResult` objects, nothing else on stdout
- ✓ Summary line: X files analysed. Y passed. Z failed.
- ✓ `--threshold` int flag (default 50): files scoring below threshold count as failed
- ✓ `--exit-code` flag: exit code 1 when any file fails threshold, exit code 0 when all pass
- ✓ `--version` flag prints package version
- ✓ `--help` flag prints usage
- ✓ Works via `dart pub global activate --source path . && bonsai analyze lib/`

## VERIFY STEP

Run `bonsai analyze --format json test/fixtures/ | python3 -m json.tool` — must parse cleanly. Run with `--threshold 80 --exit-code` against fixtures — must exit with code 1 because `heavy_screen` scores RED.

**DAY 6**  
4-5 hrs

## VS Code Extension + CI + Launch

Extension with gutter dots and hover. GitHub Actions CI. Benchmark script. README. Tag v0.1.0.

### DELIVERABLES — ALL MUST BE TRUE BEFORE MOVING ON

- ✓ extension.ts: on every .dart file save, shell out to bonsai CLI with --format json, parse result
- ✓ Green/yellow/red SVG dot gutter decoration applied to the build() method line
- ✓ HoverProvider on build() signature line shows Markdown tooltip with all three metric values and top suggestion
- ✓ StatusBar item shows score of last saved file, clicking it opens Webview analysis panel
- ✓ Warning shown if bonsai CLI is not installed with install command
- ✓ ci.yml: on push and PR — dart analyze + dart test + dart format check for both packages
- ✓ publish.yml: on v\*.\*.\* tag — dart pub publish for bonsai\_core then bonsai\_cli
- ✓ benchmark.dart: clones 5-10 public Flutter repos, runs bonsai analyze, prints markdown table of results
- ✓ README.md: install instructions, CI badge, quick-start, example output, link to pub.dev
- ✓ Tag v0.1.0 pushed

#### VERIFY STEP

*F5 in VS Code, open heavy\_screen.dart, save — red dot appears on build() line. Hover — tooltip shows blast score > 40.  
Push tag v0.1.0 — confirm CI and publish workflows trigger in GitHub Actions.*

---

## 4. Codex Prompts

8 prompts. Each one is a complete, self-contained instruction block. Copy the entire text between the dashed lines and paste it into Codex or Claude Code as-is. Do not paraphrase. Do not summarize. Do not split a prompt into parts. Each prompt builds on the previous — run them in order.

■ Before running each prompt: confirm the previous prompt's verify step passed. If dart analyze shows errors from the previous prompt, fix them before proceeding.

P1

### Repo scaffold + package skeleton

Day 1 | Est: 30-60 min

Copy the prompt block below exactly into Codex / Claude Code

#### PROMPT — COPY THIS ENTIRE BLOCK

You are building 'bonsai' — a Dart CLI + VS Code extension that statically

analyzes Flutter widget trees and produces a health score per file.

This is prompt 1 of 8. Your job is ONLY scaffolding and structure.

No logic. No implementation. Only correct package structure and empty stubs.

#### PROJECT CONTEXT:

- Tool name: bonsai
- Two Dart packages: bonsai\_core (pure Dart, the metric engine) and bonsai\_cli (the CLI)
- One VS Code extension package: bonsai\_vscode (TypeScript, created in prompt 7)
- bonsai\_core must NEVER depend on Flutter. It is pure Dart only.
- bonsai\_cli depends on bonsai\_core via path dependency

#### EXACT STEPS TO EXECUTE:

1. Create directory: bonsai/packages/
  2. Inside packages/: run `dart create --template=package bonsai\_core`
  3. Inside packages/: run `dart create --template=package bonsai\_cli`
  4. Open bonsai\_core/pubspec.yaml. Set sdk: `>=3.0.0 <4.0.0`.
- Add to dependencies: analyzer: ^6.4.0
- Add to dev\_dependencies: test: ^1.24.0
5. Open bonsai\_cli/pubspec.yaml. Set sdk: `>=3.0.0 <4.0.0`.

Add to dependencies:

bonsai\_core:

path: ../bonsai\_core

args: ^2.4.0

glob: ^2.1.0

ansi\_styles: ^0.3.0

6. In bonsai\_core/lib/src/ create these 7 empty dart files.

Each file must contain only a library declaration, nothing else:

models.dart

parser.dart

visitor.dart

scopes.dart

subtree.dart

composability.dart

```
context_detector.dart
```

```
aggregator.dart
```

7. Replace contents of `bonsai_core/lib/bonsai_core.dart` with:

```
library bonsai_core;

export 'src/models.dart';
export 'src/parser.dart';
export 'src/visitor.dart';
export 'src/scopes.dart';
export 'src/subtree.dart';
export 'src/composability.dart';
export 'src/context_detector.dart';
export 'src/aggregator.dart';
```

8. Create `bonsai_cli/bin/bonsai.dart` with content:

```
import 'package:bonsai_core/bonsai_core.dart';

void main(List args) {
  print('bonsai v0.1.0');
}
```

9. Create `bonsai/.gitignore` containing:

```
.dart_tool/
build/
.DS_Store
*.iml
.idea/
```

10. Create `bonsai/README.md` containing:

```
# bonsai

> Prune your widget trees before your users feel them.

A static analysis tool for Flutter developers.

Measures what your linter ignores. Gives every screen a score. Works in CI.

**Work in progress.** Star to follow along.
```

11. Run ``dart pub get`` in both `bonsai_core/` and `bonsai_cli/`

12. Run ``dart analyze bonsai_core/`` and ``dart analyze bonsai_cli/``

Both must show zero issues.

CONSTRAINTS – do not violate these:

- Do NOT add flutter as a dependency anywhere in any `pubspec.yaml`
- Do NOT implement any logic in this prompt
- Library names in `pubspec` must be exactly: `bonsai_core` and `bonsai_cli`
- Minimum SDK must be exactly: `'>=3.0.0 <4.0.0'`
- Empty dart files must still be valid dart – use ``library filename;`` as content

#### EXPECTED OUTPUT — WHAT DONE LOOKS LIKE

- ✓ `dart pub get` succeeds in both packages with zero warnings
- ✓ `dart analyze` shows zero issues on both packages
- ✓ All 8 `src/` files exist and are valid Dart libraries
- ✓ No flutter dependency anywhere in the dependency tree

✓ bonsai\_cli/bin/bonsai.dart compiles and prints 'bonsai v0.1.0'

#### VERIFY BEFORE RUNNING NEXT PROMPT

Run: `cd bonsai/packages/bonsai_core && dart pub get` Run: `cd bonsai/packages/bonsai_cli && dart pub get` Run: `dart run bonsai/packages/bonsai_cli/bin/bonsai.dart` — must print `bonsai v0.1.0`

P2

## Data models — FileMetrics, HealthScore, AnalysisResult

Day 1 | Est: 45 min

Copy the prompt block below exactly into Codex / Claude Code

#### PROMPT — COPY THIS ENTIRE BLOCK

You are working on 'bonsai', a Flutter static analysis tool.

This is prompt 2 of 8. Previous prompt completed the repo scaffold.

Your job is to implement ALL data models in models.dart.

No visitors. No parsing. Pure model definitions only.

FILE TO EDIT: `bonsai/packages/bonsai_core/lib/src/models.dart`

IMPLEMENT EXACTLY THE FOLLOWING — in this order:

1. ENUM ScopeType:

```
enum ScopeType { setState, streamBuilder, consumer, valueListenable,
blocBuilder, selector, futureBuilder, animatedBuilder, other }
```

2. CLASS ScopeInfo:

```
final fields: ScopeType type, int line, int column,
int widgetCount, int nestingDepth
const constructor with all fields required
No methods needed.
```

3. CLASS ContextCall:

```
final fields: String callType, int nestingDepth, int line
(callType is one of: 'watch', 'read', 'select', 'provider_of')
const constructor with all fields required
```

4. CLASS FileMetrics:

```
final fields:
String filePath
List scopes
int maxBlastScore (max of all scope.widgetCount values, 0 if no scopes)
double composabilityRatio (double between 0.0 and 1.0)
int maxContextDepth (max nesting depth of any context call, 0 if none)
int leafWidgetCount
int extractedWidgetCount
bool parseError (true if file failed to parse, default false)
const constructor. parseError defaults to false.
Factory: FileMetrics.empty(String filePath) — all numeric fields 0,
composabilityRatio 1.0, empty lists, parseError false.
```

Factory: FileMetrics.parseFailure(String filePath) – same as empty  
but parseError = true.

#### 5. CLASS HealthScore:

final fields: int score, String band, List suggestions

score must be clamped to 0-100

band must be exactly 'GREEN', 'YELLOW', or 'RED' – no other values

const constructor.

Factory: HealthScore.fromMetrics(FileMetrics m):

Implement this formula exactly:

double penalty = 0;

penalty += (m.maxBlastScore / 2).clamp(0, 40);

penalty += ((1 - m.composabilityRatio) \* 30).clamp(0, 30);

penalty += (m.maxContextDepth \* 3).clamp(0, 30);

int score = (100 - penalty).round().clamp(0, 100);

String band = score >= 75 ? 'GREEN' : score >= 50 ? 'YELLOW' : 'RED';

Generate suggestions list based on which penalty was highest:

If blast penalty > 20: add 'Extract widgets inside reactive scopes – blast score is HIGH'

If composability penalty > 15: add 'Decompose build() into smaller custom widget classes'

If context depth penalty > 15: add 'Move context.watch/read calls closer to the root of build()'

toString(): return 'score/100 [band]' e.g. '31/100 [RED]'

#### 6. CLASS AnalysisResult:

final fields: FileMetrics metrics, HealthScore health

const constructor.

Method: Map toJson() returning:

```
{  
  'file': metrics.filePath,  
  'score': health.score,  
  'band': health.band,  
  'blastScore': metrics.maxBlastScore,  
  'composability': metrics.composabilityRatio,  
  'contextDepth': metrics.maxContextDepth,  
  'suggestions': health.suggestions,  
  'parseError': metrics.parseError,  
}
```

#### CONSTRAINTS:

- All fields must be final
- Use @immutable annotation on all classes (import package:meta/meta.dart
- add meta: ^1.9.0 to bonsai\_core dev\_dependencies)
- No external packages other than meta in this file
- Do not use json\_serializable or any code gen
- Division by zero guard: if leafWidgetCount + extractedWidgetCount == 0,

composabilityRatio should be treated as 1.0 (no leaves = perfectly decomposed)

#### EXPECTED OUTPUT — WHAT DONE LOOKS LIKE

- ✓ All 5 classes and 1 enum defined with correct types
- ✓ HealthScore.fromMetrics(FileMetrics with blast=60) returns score < 50 and band 'RED'
- ✓ HealthScore.fromMetrics(FileMetrics with blast=5, composability=0.8, depth=2) returns band 'GREEN'
- ✓ toJson() returns a flat Map with all 8 keys
- ✓ dart analyze shows zero issues

#### VERIFY BEFORE RUNNING NEXT PROMPT

Write a temporary main() at bottom of models.dart, create two FileMetrics manually and print HealthScore.fromMetrics for each. One must be RED, one GREEN. Remove main() after confirming.

P3

### AST visitor — reactive scope detection + blast radius

Day 3 | Est: 3-4 hrs

Copy the prompt block below exactly into Codex / Claude Code

#### PROMPT — COPY THIS ENTIRE BLOCK

You are working on 'bonsai', a Flutter static analysis tool.

This is prompt 3 of 8. The hardest prompt. Take your time.

Previous prompts completed scaffold and models.

Your job: implement the AST parser and the BonsaiVisitor that detects reactive scopes and counts widget subtrees.

#### FILES TO IMPLEMENT:

bonsai/packages/bonsai\_core/lib/src/scopes.dart

bonsai/packages/bonsai\_core/lib/src/subtree.dart

bonsai/packages/bonsai\_core/lib/src/visitor.dart

bonsai/packages/bonsai\_core/lib/src/parser.dart

#### BACKGROUND — HOW THE ANALYZER PACKAGE WORKS:

The 'analyzer' package (already in pubspec) parses Dart source into an AST.

Key import: import 'package:analyzer/dart/analysis/utilities.dart';

Key function: parseString(content: String, featureSet: FeatureSet)

returns ParseStringResult with .unit of type CompilationUnit.

You walk the AST by extending RecursiveAstVisitor and overriding

visit methods. The visitor calls visitChildren() automatically if you

call super.visitXxx(node) — do NOT call node.visitChildren(visitor) manually.

Key AST node types:

MethodDeclaration — a method. Check node.name.lexeme == 'build'

InstanceCreationExpression — calling a constructor e.g. Text('hello')

Access constructor name via: node.constructorName.type.name2.lexeme

MethodInvocation — calling a method e.g. setState(() {})

Check: node.methodName.name == 'setState'

The 'builder' argument of StreamBuilder etc is a FunctionExpression inside

the named argument. You need to count widgets inside the entire node subtree.

IMPLEMENT scopes.dart:

```

const List kReactiveScopes = [
  'StreamBuilder', 'Consumer', 'BlocBuilder', 'ValueListenableBuilder',
  'Selector', 'FutureBuilder', 'AnimatedBuilder', 'Consumer2',
  'BlocConsumer', 'GetBuilder', 'Obx', 'GetX',
];

const List kReactiveMethods = ['setState'];

IMPLEMENT subtree.dart:
import 'package:analyzer/dart/ast/ast.dart';

int countWidgetsInSubtree(AstNode root) {
  Implement a _CountingVisitor extends RecursiveAstVisitor that:
  - In visitInstanceCreationExpression(node): checks if the constructor
  name starts with an uppercase letter (it is a widget or widget-like class).
  Use: node.constructorName.type.name2.lexeme[0].
  If it starts with uppercase: increment a counter.
  Always call super.visitInstanceCreationExpression(node) to recurse.
  Return the counter value after visiting root.
}

IMPLEMENT visitor.dart:
class BonsaiVisitor extends RecursiveAstVisitor {
  Fields:
  final List _scopes = [];
  int _currentDepth = 0; // widget nesting depth inside build()
  bool _inBuildMethod = false; // true when inside a build() method

  Override visitMethodDeclaration(MethodDeclarationImpl node):
  If node.name.lexeme == 'build':
  Set _inBuildMethod = true, _currentDepth = 0
  Call super (recurses into the method body)
  Set _inBuildMethod = false
  Else: call super

  Override visitInstanceCreationExpression(node):
  If _inBuildMethod:
  String name = node.constructorName.type.name2.lexeme
  If name is in kReactiveScopes:
  int count = countWidgetsInSubtree(node)
  _scopes.add(ScopeInfo(
    type: _scopeTypeFrom(name),
    line: node.offset, // use offset as line approximation
    column: 0,
    widgetCount: count,
    nestingDepth: _currentDepth,
  ))

```

```

    _currentDepth++
    super.visitInstanceCreationExpression(node)
    _currentDepth--
Else: super.visitInstanceCreationExpression(node)

Override visitMethodInvocation(node):
If _inBuildMethod && node.methodName.name is in kReactiveMethods:
int count = countWidgetsInSubtree(node)
_scopes.add(ScopeInfo(
    type: ScopeType.setState,
    line: node.offset,
    column: 0,
    widgetCount: count,
    nestingDepth: _currentDepth,
))
Always call super.

Getter List get scopes => List.unmodifiable(_scopes)

Getter int get maxBlastScore =>
_scopes.isEmpty ? 0 : _scopes.map((s) => s.widgetCount).reduce(max)

Private helper _scopeTypeFrom(String name) -> ScopeType:
Maps name strings to ScopeType enum values.
Default: ScopeType.other
}

IMPLEMENT parser.dart:
import 'package:analyzer/dart/analysis/utilities.dart';
import 'package:analyzer/dart/analysis/features.dart';
import 'package:analyzer/src/dart/analysis/experiments.dart';

CompilationUnit? parseFile(String path):
Read file content with File(path).readAsStringSync()
Wrap in try-catch. On error return null.
Call parseString(content: content, featureSet: FeatureSet.latestLanguageVersion())
Return result.unit

FileMetrics analyzeFile(String path):
CompilationUnit? unit = parseFile(path)
If unit is null: return FileMetrics.parseFailure(path)
BonsaiVisitor visitor = BonsaiVisitor()
unit.visitChildren(visitor)
return FileMetrics(
    filePath: path,
    scopes: visitor.scopes,
    maxBlastScore: visitor.maxBlastScore,
    composabilityRatio: 1.0, // placeholder until prompt 4

```

```
maxContextDepth: 0, // placeholder until prompt 4
leafWidgetCount: 0, // placeholder until prompt 4
extractedWidgetCount: 0, // placeholder until prompt 4
)
```

#### CONSTRAINTS:

- Import dart:math for max() function
- Do NOT use resolveFile or AnalysisContext – use parseString only (parse-only mode)
- Do NOT throw exceptions from analyzeFile – always return a FileMetrics
- The \_currentDepth counter must increment BEFORE calling super and decrement AFTER – this is critical for correct depth tracking
- For setState, count widgets inside its first argument (the callback body), not including the setState call itself
- A widget is any InstanceCreationExpression where constructor name starts with an uppercase letter

CREATE TWO FIXTURE FILES for testing:

bonsai/packages/bonsai\_core/test/fixtures/heavy\_screen.dart:

Write a realistic Flutter-looking file (no imports needed, just valid Dart)

with a build() method containing:

A StreamBuilder wrapping a Column with 20 Text widgets

And a Consumer wrapping a Row with 15 Icon widgets

This should produce maxBlastScore around 20-22.

bonsai/packages/bonsai\_core/test/fixtures/clean\_screen.dart:

A build() method that returns a CustomAppBar widget,

which contains a ProfileSection and a SettingsButton.

No reactive scopes. This should produce maxBlastScore of 0.

#### EXPECTED OUTPUT — WHAT DONE LOOKS LIKE

- ✓ analyzeFile on heavy\_screen.dart returns maxBlastScore between 18 and 25
- ✓ analyzeFile on clean\_screen.dart returns maxBlastScore of 0
- ✓ analyzeFile on a file with syntax errors returns parseError: true without throwing
- ✓ dart analyze shows zero issues on all four files

#### VERIFY BEFORE RUNNING NEXT PROMPT

Create a temp test\_run.dart in bonsai\_core/, import bonsai\_core.dart, call analyzeFile on both fixtures, print the FileMetrics. Confirm blast scores match expectations. Delete test\_run.dart after confirming.

**P4**

### Composability scorer + context coupling + full test suite

Day 4 | Est: 3-4 hrs

Copy the prompt block below exactly into Codex / Claude Code

#### PROMPT — COPY THIS ENTIRE BLOCK

You are working on 'bonsai', a Flutter static analysis tool.

This is prompt 4 of 8. Previous prompts completed models, constants, and blast score.

Your job: add the remaining two metrics (composability + context depth)

and write the complete unit and integration test suite.

FILES TO IMPLEMENT / EDIT:

bonsai/packages/bonsai\_core/lib/src/composability.dart (new)

bonsai/packages/bonsai\_core/lib/src/context\_detector.dart (new)

bonsai/packages/bonsai\_core/lib/src/visitor.dart (edit – add leaf/custom counting)

bonsai/packages/bonsai\_core/lib/src/parser.dart (edit – merge all metrics)

bonsai/packages/bonsai\_core/test/unit/models\_test.dart (new)

bonsai/packages/bonsai\_core/test/unit/blast\_test.dart (new)

bonsai/packages/bonsai\_core/test/unit/composability\_test.dart (new)

bonsai/packages/bonsai\_core/test/unit/context\_test.dart (new)

bonsai/packages/bonsai\_core/test/integration/analyze\_file\_test.dart (new)

IMPLEMENT composability.dart:

```
const List kLeafWidgets = [
  'Text', 'Icon', 'Image', 'NetworkImage', 'AssetImage',
  'SizedBox', 'Container', 'Padding', 'Divider', 'VerticalDivider',
  'Center', 'Align', 'Expanded', 'Flexible', 'Spacer',
  'CircularProgressIndicator', 'LinearProgressIndicator',
  'IconButton', 'TextButton', 'ElevatedButton', 'OutlinedButton',
  'TextField', 'TextFormField', 'Checkbox', 'Switch', 'Radio',
  'Tooltip', 'Ink', 'InkWell', 'GestureDetector',
];
```

double computeComposability(int leafCount, int extractedCount):

If leafCount + extractedCount == 0: return 1.0

Return extractedCount / (leafCount + extractedCount).toDouble()

Result must be between 0.0 and 1.0.

EDIT visitor.dart – add to BonsaiVisitor:

Add two new int fields: \_leafCount = 0, \_extractedCount = 0

In visitInstanceCreationExpression, AFTER the reactive scope check:

String name = node.constructorName.type.name2.lexeme

If name is in kLeafWidgets: \_leafCount++

Else if name is NOT in kReactiveScopes AND name starts with uppercase:

\_extractedCount++

Add getters:

int get leafWidgetCount => \_leafCount

int get extractedWidgetCount => \_extractedCount

double get composabilityRatio => computeComposability(\_leafCount, \_extractedCount)

IMPLEMENT context\_detector.dart:

class ContextCallDetector extends RecursiveAstVisitor:

Fields:

final List \_calls = [];

int \_currentDepth = 0;

```
bool _inBuildMethod = false;
```

```
Override visitMethodDeclaration(node):
```

```
If node.name.lexeme == 'build':
```

```
  _inBuildMethod = true; _currentDepth = 0
```

```
  super.visitMethodDeclaration(node)
```

```
  _inBuildMethod = false
```

```
Else: super
```

```
Override visitInstanceCreationExpression(node):
```

```
If _inBuildMethod: _currentDepth++; super; _currentDepth--
```

```
Else: super
```

```
Override visitMethodInvocation(node):
```

```
If _inBuildMethod:
```

```
  String target = node.target?.toSource() ?? '';
```

```
  String method = node.methodName.name;
```

```
  bool isContextCall = target == 'context' &&
```

```
    (method == 'watch' || method == 'read' || method == 'select');
```

```
  bool isProviderOf = target == 'Provider' && method == 'of';
```

```
If isContextCall:
```

```
  _calls.add(ContextCall(callType: method, nestingDepth: _currentDepth, line: node.offset))
```

```
If isProviderOf:
```

```
  _calls.add(ContextCall(callType: 'provider_of', nestingDepth: _currentDepth, line: node.offset))
```

```
Always call super.
```

```
Getter List get calls => List.unmodifiable(_calls)
```

```
Getter int get maxDepth => _calls.isEmpty ? 0 : _calls.map((c) => c.nestingDepth).reduce(max)
```

```
EDIT parser.dart – update analyzeFile():
```

```
After running BonsaiVisitor, also run ContextCallDetector on the same unit.
```

```
Build final FileMetrics with all fields populated:
```

```
composabilityRatio: visitor.composabilityRatio,
```

```
maxContextDepth: contextDetector.maxDepth,
```

```
leafWidgetCount: visitor.leafWidgetCount,
```

```
extractedWidgetCount: visitor.extractedWidgetCount,
```

```
CREATE FIXTURE FILES:
```

```
test/fixtures/deep_context.dart:
```

```
A build() method with Widget structure nested 8 levels deep,
```

```
and a context.watch call at the innermost level.
```

```
Expected: maxContextDepth == 8 (approximately)
```

```
test/fixtures/broken_syntax.dart:
```

```
A file with intentionally broken Dart: `class Foo { void bar( `
```

```
Expected: parseError == true
```

```
WRITE TEST FILES:
```

```
test/unit/models_test.dart:
```

```
group('HealthScore.fromMetrics', () {  
  Test 1: blast=80, composability=0.0, depth=10 -> score < 10, band RED  
  Test 2: blast=0, composability=1.0, depth=0 -> score == 100, band GREEN  
  Test 3: blast=40, composability=0.5, depth=5 -> band YELLOW  
  Test 4: blast=5, composability=0.9, depth=1 -> band GREEN  
  Test 5: FileMetrics.parseFailure -> parseError == true  
  Test 6: toJson() has exactly 8 keys and all expected key names  
})
```

```
test/unit/blast_test.dart:
```

```
group('countWidgetsInSubtree', () {  
  Parse inline Dart strings using parseString(), find the first  
  InstanceCreationExpression node, call countWidgetsInSubtree on it.  
  Test 1: StreamBuilder with Column and 5 Text -> count around 7  
  Test 2: Single Text() -> count == 1  
  Test 3: Empty Container() -> count == 1  
  Test 4: Deeply nested Column > Row > Expanded > Text x3 -> count == 6  
})
```

```
test/unit/composability_test.dart:
```

```
group('computeComposability', () {  
  Test 1: leafCount=0, extractedCount=0 -> returns 1.0  
  Test 2: leafCount=10, extractedCount=0 -> returns 0.0  
  Test 3: leafCount=0, extractedCount=5 -> returns 1.0  
  Test 4: leafCount=6, extractedCount=4 -> returns 0.4  
  Test 5: result is always between 0.0 and 1.0 inclusive  
})
```

```
test/unit/context_test.dart:
```

```
group('ContextCallDetector', () {  
  Use parseString() to parse inline Dart, run ContextCallDetector on unit.  
  Test 1: no context calls -> maxDepth == 0, calls empty  
  Test 2: context.watch at top of build -> maxDepth == 0 or 1  
  Test 3: context.watch nested inside 3 widgets -> maxDepth == 3  
  Test 4: Provider.of call -> callType == 'provider_of'  
  Test 5: context.read and context.watch both present -> both detected  
})
```

```
test/integration/analyze_file_test.dart:
```

```
group('analyzeFile integration', () {  
  Test 1: heavy_screen.dart -> health.band == 'RED' or 'YELLOW'  
  Test 2: clean_screen.dart -> health.band == 'GREEN'  
  Test 3: broken_syntax.dart -> metrics.parseError == true  
  Test 4: deep_context.dart -> metrics.maxContextDepth > 5  
})
```

#### CONSTRAINTS:

- Every test must have a descriptive name that reads as a sentence
- Use `expect()` not `assert()`
- No golden files – all assertions explicit
- Do not mock the AST – parse real inline Dart strings in unit tests
- Run dart test at end – must show 0 failures
- Run with coverage: `dart test --coverage=coverage/`

Coverage on `lib/src/` must be `>= 85%`

#### EXPECTED OUTPUT — WHAT DONE LOOKS LIKE

- ✓ dart test shows 0 failures across all 5 test files
- ✓ `heavy_screen.dart` produces RED or YELLOW band
- ✓ `clean_screen.dart` produces GREEN band
- ✓ `broken_syntax.dart` produces `parseError: true`
- ✓ Coverage on `lib/src/` `>= 85%`

#### VERIFY BEFORE RUNNING NEXT PROMPT

*Run: `dart test --reporter=expanded` Run: `dart test --coverage=coverage/ && dart run coverage:format_coverage --lcov` Every group must show **PASSED**. Check coverage report.*

P5

### CLI — bonsai analyze command

Day 5 | Est: 2-3 hrs

Copy the prompt block below exactly into Codex / Claude Code

#### PROMPT — COPY THIS ENTIRE BLOCK

You are working on 'bonsai', a Flutter static analysis tool.

This is prompt 5 of 8. The metric engine is complete and tested.

Your job: implement the full CLI so users can run

`bonsai analyze lib/`

and get colour-coded output or JSON output.

#### FILES TO IMPLEMENT:

`bonsai/packages/bonsai_cli/bin/bonsai.dart` (replace stub)

`bonsai/packages/bonsai_cli/lib/src/runner.dart` (new)

`bonsai/packages/bonsai_cli/lib/src/formatter.dart` (new)

`bonsai/packages/bonsai_cli/lib/src/config.dart` (new)

#### IMPLEMENT `runner.dart`:

```
import 'package:args/args.dart';
```

```
import 'package:glob/glob.dart';
```

```
import 'package:glob/list_local_fs.dart';
```

```
import 'package:bonsai_core/bonsai_core.dart';
```

```
class BonsaiRunner:
```

```
Method: Future run(List args)
```

```
Returns exit code: 0 = all pass, 1 = some fail, 2 = fatal error
```

```
Parse args using ArgParser:
```

```

'format' option: allowed ['pretty', 'json'], default 'pretty'
'threshold' option: int, default 50, help: 'Min score to pass'
'exit-code' flag: bool, default false
'version' flag: bool, default false
'help' flag: bool, default false
rest: positional path, default 'lib'

If version flag: print '■ bonsai v0.1.0', return 0
If help flag: print usage string, return 0

String targetPath = args.rest.isEmpty ? 'lib' : args.rest.first
If directory does not exist: print error, return 2

Use Glob('*.dart') to find all .dart files under targetPath.
Filter out files ending in '.g.dart' or '.freezed.dart'.

For each file: call analyzeFile(path) -> AnalysisResult
Collect all results.

If format == 'json': call JsonFormatter().format(results), print, return 0
If format == 'pretty': call PrettyFormatter().format(results, threshold)

int passed = results.where((r) => r.health.score >= threshold).length
int failed = results.length - passed

If format == 'pretty':
print summary: 'Analysed X files. Y passed. Z failed (below threshold T).'

If exitCode flag && failed > 0: return 1
Return 0

IMPLEMENT formatter.dart:

class PrettyFormatter:
void format(List results, int threshold):
For each result:
String filename = basename(result.metrics.filePath)
Pad filename to 45 characters with spaces (truncate if longer)
String scoreStr = result.health.score.toString().padLeft(3) + '/100'
String band = result.health.band // 'GREEN', 'YELLOW', 'RED'

Apply ANSI colour codes:
GREEN: '■[32m' + text + '■[0m'
YELLOW: '■[33m' + text + '■[0m'
RED: '■[31m' + text + '■[0m'

Print one line: filename + ' ' + coloured(scoreStr + ' ' + band)

If result.health.score < threshold:
For each suggestion in result.health.suggestions:
Print ' -> ' + suggestion (indented, no colour)

```

```
If result.metrics.parseError:
```

```
Print ' [!] Could not parse file – check for syntax errors'
```

```
class JsonFormatter:
```

```
String format(List results):
```

```
Convert each result to Map using result.toJson()
```

```
Return JSON-encoded array using dart:convert jsonEncode()
```

```
Must be PURE JSON – no colour codes, no extra text, no newlines before array
```

```
IMPLEMENT config.dart:
```

```
(Minimal – just load threshold from .bonsai.yaml if present)
```

```
class BonsaiConfig:
```

```
final int threshold;
```

```
final List excludePaths;
```

```
static BonsaiConfig load(String workingDir):
```

```
Look for .bonsai.yaml in workingDir.
```

```
If not found: return BonsaiConfig(threshold: 50, excludePaths: [])
```

```
If found: parse YAML. Read 'threshold' int and 'exclude_paths' list.
```

```
Note: add yaml: ^3.1.0 to bonsai_cli pubspec dependencies for this.
```

```
Return BonsaiConfig with parsed values.
```

```
IMPLEMENT bonsai.dart (replace stub):
```

```
import the runner
```

```
void main(List args) async {
```

```
final runner = BonsaiRunner();
```

```
final exitCode = await runner.run(args);
```

```
exit(exitCode); // import dart:io for exit()
```

```
}
```

```
UPDATE bonsai_cli/pubspec.yaml:
```

```
Add yaml: ^3.1.0 to dependencies.
```

```
CONSTRAINTS:
```

```
- JSON output must be PURE JSON – nothing on stdout before or after the array
```

```
- Generated file exclusion is mandatory: skip *.g.dart AND *.freezed.dart
```

```
- ANSI codes must be actual escape codes (■[32m etc), not emoji
```

```
- If a file fails to parse, still include it in results with parseError flag
```

```
- Default threshold is 50 – this means scores of 50 and above PASS
```

```
- Exit code semantics: 0 = all pass OR exitCode flag not set,
```

```
1 = some fail AND exitCode flag set, 2 = fatal error (bad path etc)
```

```
- The CLI must work via: dart pub global activate --source path .
```

```
then: bonsai analyze lib/
```

```
VERIFY COMMANDS TO RUN:
```

```
dart run bin/bonsai.dart analyze ../bonsai_core/test/fixtures/
```

```
dart run bin/bonsai.dart analyze --format json ../bonsai_core/test/fixtures/
```

```
dart run bin/bonsai.dart --version
dart run bin/bonsai.dart --help
dart run bin/bonsai.dart analyze --threshold 90 --exit-code ../bonsai_core/test/fixtures/
(last command should exit with code 1 since heavy_screen will score below 90)
```

#### EXPECTED OUTPUT — WHAT DONE LOOKS LIKE

- ✓ bonsai analyze fixtures/ prints colour-coded table with correct scores
- ✓ bonsai analyze --format json fixtures/ produces valid JSON parseable by python3 -m json.tool
- ✓ --exit-code flag returns exit code 1 when heavy\_screen is in path with threshold 80+
- ✓ --version and --help both work
- ✓ Generated .g.dart files are excluded from analysis

#### VERIFY BEFORE RUNNING NEXT PROMPT

Run all 5 verify commands above. Run the JSON output through `python3 -m json.tool` to confirm it parses. Check exit codes with `$?` after each command.

P6

### VS Code extension — gutter scores + hover tooltip

Day 6 | Est: 3-4 hrs

Copy the prompt block below exactly into Codex / Claude Code

#### PROMPT — COPY THIS ENTIRE BLOCK

```
You are working on 'bonsai', a Flutter static analysis tool.

This is prompt 6 of 8. The CLI is complete.

Your job: implement the VS Code extension that shows the health score
as a gutter decoration and hover tooltip on every Flutter file as you code.

The extension shells out to the bonsai CLI – no Dart analysis in TypeScript.

SCAFFOLD THE EXTENSION:

In bonsai/packages/bonsai_vscode/:

Run: npm install -g yo generator-code (if not installed)

Run: yo code

Choose: TypeScript, name: bonsai, no webpack, yes to git

Or manually create the structure:

src/extension.ts
src/cli_runner.ts
src/webview.ts
package.json
tsconfig.json
media/dot-green.svg
media/dot-yellow.svg
media/dot-red.svg

CREATE MEDIA FILES:

media/dot-green.svg:

media/dot-yellow.svg: same with fill='#E8A020'

media/dot-red.svg: same with fill='#C0392B'
```

```
IMPLEMENT src/cli_runner.ts:
```

```
import * as cp from 'child_process';
```

```
export interface BonsaiResult {
```

```
file: string;
```

```
score: number;
```

```
band: 'GREEN' | 'YELLOW' | 'RED';
```

```
blastScore: number;
```

```
composability: number;
```

```
contextDepth: number;
```

```
suggestions: string[];
```

```
parseError: boolean;
```

```
}
```

```
export async function runBonsai(filePath: string): Promise:
```

```
Run: bonsai analyze --format json as a child process
```

```
Timeout after 8000ms – if timeout, return null
```

```
If exit code is 2 (fatal): return null
```

```
If bonsai command not found: return null (will show install warning)
```

```
Parse stdout as JSON array, find the entry matching filePath
```

```
Return it or null if not found
```

```
Use child_process.spawn (not exec) to avoid buffer limits.
```

```
Wrap in Promise that resolves on 'close' event.
```

```
On any error, return null – never throw.
```

```
export function isBonsaiInstalled(): boolean:
```

```
Try running: bonsai --version synchronously
```

```
Return true if exit code is 0
```

```
Return false on any error
```

```
IMPLEMENT src/extension.ts:
```

```
import * as vscode from 'vscode';
```

```
import { runBonsai, isBonsaiInstalled } from './cli_runner';
```

```
export function activate(context: vscode.ExtensionContext):
```

```
1. CREATE THREE DECORATION TYPES:
```

```
const greenDot = vscode.window.createTextEditorDecorationType({
```

```
gutterIconPath: context.asAbsolutePath('media/dot-green.svg'),
```

```
gutterIconSize: 'contain',
```

```
})
```

```
Similarly for yellow and red.
```

```
2. CREATE STATUS BAR ITEM:
```

```
const statusBar = vscode.window.createStatusBarItem(
```

```
vscode.StatusBarAlignment.Right, 100)
```

```
statusBar.text = '■ bonsai'
```

```

statusBar.tooltip = 'Click to analyze workspace'
statusBar.command = 'bonsai.analyzeWorkspace'
statusBar.show()

3. ASYNC FUNCTION analyzeCurrentFile(editor: vscode.TextEditor):
If editor.document.languageId !== 'dart': return
If !isBonsaiInstalled():
statusBar.text = '■ bonsai: not installed'
statusBar.tooltip = 'Run: dart pub global activate bonsai'
Return

const result = await runBonsai(editor.document.fileName)
If result is null: return

Find line number of 'Widget build(' in the document:
Loop through document lines, find first line containing 'Widget build('
Use that line number. If not found, use line 0.

const range = new vscode.Range(buildLineNumber, 0, buildLineNumber, 0)
const decoration = { range }

Clear all three decoration types on editor first.
Apply the matching decoration based on result.band.

statusBar.text = `■ bonsai: ${result.score}/100`
statusBar.backgroundColor = result.band === 'RED'
? new vscode.ThemeColor('statusBarItem.errorBackground') : undefined

4. REGISTER HOVER PROVIDER:
vscode.languages.registerHoverProvider('dart', {
provideHover(document, position) {
const line = document.lineAt(position.line).text
If line does not contain 'Widget build(': return null

Run runBonsai synchronously (use cached last result or re-run)
Note: hovers can be async – return a Promise

Build Markdown string:
## ■ bonsai health score
**Score:** result.score/100 [result.band]
---
| Metric | Value |
|-----|-----|
| Blast Score | result.blastScore widgets |
| Composability | (result.composability * 100).toFixed(0)% |
| Context Depth | result.contextDepth |
---
**Suggestions:**
- suggestion1

```

```
- suggestion2
```

```
Return new vscode.Hover(new vscode.MarkdownString(markdown))
}
})
```

5. REGISTER ON-SAVE TRIGGER:

```
vscode.workspace.onDidSaveTextDocument(async (document) => {
If document.languageId !== 'dart': return
const editor = vscode.window.visibleTextEditors.find(
e => e.document === document)
If editor: await analyzeCurrentFile(editor)
})
```

6. REGISTER COMMAND bonsai.analyzeWorkspace:

```
Run bonsai analyze lib/ --format json on workspace root
Parse results into a Webview panel showing an HTML table
See webview.ts for the panel implementation
```

7. Trigger analysis on the currently active editor on activation:

```
If vscode.window.activeTextEditor exists and is a dart file:
analyzeCurrentFile(vscode.window.activeTextEditor)
```

```
export function deactivate(): clear all decorations and statusBar.
```

IMPLEMENT src/webview.ts:

```
export function showWorkspacePanel(context, results: BonsaiResult[]):
```

Create a WebviewPanel with title 'bonsai: Workspace Analysis'

Set HTML content with:

A table showing: filename, score, band, blast score, composability, context depth

Rows colour-coded: green bg for GREEN, yellow bg for YELLOW, red bg for RED

A summary line: X files analysed. Y passed. Z failed.

Use CSP: default-src 'none'; style-src 'unsafe-inline'

No external scripts.

IMPLEMENT package.json:

```
name: bonsai, displayName: 'bonsai', publisher: your-publisher-id
```

```
engines: { vscode: '^1.85.0' }
```

```
activationEvents: ['onLanguage:dart']
```

```
contributes:
```

```
commands: [{ command: 'bonsai.analyzeWorkspace', title: 'bonsai: Analyze Workspace' }]
```

```
main: ./out/extension.js
```

CONSTRAINTS:

- Never run Dart analysis in TypeScript – always shell out to bonsai CLI
- All CLI calls must be async and have an 8-second timeout
- Gutter icons must be SVG files from media/ directory – not text characters
- The hover Markdown must use a table – not a list of key:value lines

- If bonsai is not installed, show a clear message in status bar – do not throw
- Compile with: `npm run compile (tsc -p .)` before testing
- Test with F5 Extension Development Host – not unit tests

#### VERIFY STEPS:

1. Run `npm run compile` – zero TypeScript errors
2. Press F5 – Extension Development Host opens
3. Open any `.dart` file from the fixtures directory
4. Save the file – gutter dot must appear on the `'Widget build('` line
5. Hover over `'Widget build('` – tooltip table must appear with all metrics
6. Status bar must show score
7. Run Command Palette: `bonsai: Analyze Workspace` – panel must open

#### EXPECTED OUTPUT — WHAT DONE LOOKS LIKE

- ✓ `npm run compile` produces zero TypeScript errors
- ✓ Gutter dot appears on `Widget build()` line after saving a `.dart` file
- ✓ Hover tooltip shows score table with all three metrics
- ✓ Status bar shows score of last saved file
- ✓ Workspace panel opens from command palette

#### VERIFY BEFORE RUNNING NEXT PROMPT

*F5 to open Extension Development Host. Open `heavy_screen.dart` from fixtures. Save. Confirm red dot on `build()` line. Hover — confirm tooltip. Open `clean_screen.dart`. Save. Confirm green dot.*

P7

## GitHub Actions CI + benchmark script + README

Day 6 | Est: 1-2 hrs

Copy the prompt block below exactly into Codex / Claude Code

#### PROMPT — COPY THIS ENTIRE BLOCK

You are working on 'bonsai', a Flutter static analysis tool.

This is prompt 7 of 8. CLI and VS Code extension are complete.

Your job: wire up GitHub Actions CI, write the benchmark script that runs bonsai against real Flutter repos, and write the final README.

#### FILES TO CREATE:

`bonsai/.github/workflows/ci.yml`

`bonsai/.github/workflows/publish.yml`

`bonsai/scripts/benchmark.dart`

`bonsai/README.md` (final version)

`bonsai/CONTRIBUTING.md`

#### IMPLEMENT `.github/workflows/ci.yml`:

```
name: CI

on:

push:

branches: [ main ]

pull_request:

branches: [ main ]
```

```
jobs:
test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: dart-lang/setup-dart@v1
with:
sdk: stable
- name: Install dependencies (core)
working-directory: packages/bonsai_core
run: dart pub get
- name: Install dependencies (cli)
working-directory: packages/bonsai_cli
run: dart pub get
- name: Analyze (core)
working-directory: packages/bonsai_core
run: dart analyze
- name: Analyze (cli)
working-directory: packages/bonsai_cli
run: dart analyze
- name: Test (core)
working-directory: packages/bonsai_core
run: dart test
- name: Format check (core)
working-directory: packages/bonsai_core
run: dart format --output=none --set-exit-if-changed .
- name: Format check (cli)
working-directory: packages/bonsai_cli
run: dart format --output=none --set-exit-if-changed .
```

IMPLEMENT .github/workflows/publish.yml:

```
name: Publish
on:
push:
tags:
- 'v*.*.*'
jobs:
publish:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: dart-lang/setup-dart@v1
with:
sdk: stable
```

```
- name: Setup pub credentials
```

```
run: |
```

```
mkdir -p ~/.config/dart
```

```
echo '${ secrets.PUB_CREDENTIALS }' > ~/.config/dart/pub-credentials.json
```

```
- name: Publish bonsai_core
```

```
working-directory: packages/bonsai_core
```

```
run: dart pub publish --force
```

```
- name: Publish bonsai_cli
```

```
working-directory: packages/bonsai_cli
```

```
run: dart pub publish --force
```

```
IMPLEMENT scripts/benchmark.dart:
```

```
A standalone Dart script (no package, runs with dart run).
```

```
Define a list of 8 public Flutter repos to benchmark:
```

```
const repos = [
```

```
{'name': 'flutter/gallery', 'url': 'https://github.com/flutter/gallery.git', 'lib': 'lib'},
```

```
{'name': 'mitesh77/Best-Flutter-UI-Templates', 'url':  
'https://github.com/mitesh77/Best-Flutter-UI-Templates.git', 'lib': 'lib'},
```

```
{'name': 'iampawan/FlutterExampleApps', 'url': 'https://github.com/iampawan/FlutterExampleApps.git',  
'lib': 'lib'},
```

```
{'name': 'abuanwar072/UI-Designs', 'url': 'https://github.com/abuanwar072/UI-Designs.git', 'lib':  
'lib'},
```

```
// add 4 more real public Flutter repos
```

```
];
```

```
For each repo:
```

```
1. Clone to /tmp/bonsai_bench_ using Process.run('git', ['clone', '--depth=1', url, dir])
```

```
2. If clone fails: print 'SKIP : clone failed', continue
```

```
3. Run: Process.run('dart', ['pub', 'global', 'run', 'bonsai', 'analyze', libPath, '--format',  
'json'])
```

```
in the cloned directory
```

```
4. Parse JSON output (it is a JSON array)
```

```
5. Compute:
```

```
int fileCount = results.length
```

```
double avgScore = results.map((r) => r['score']).average
```

```
String lowestFile = results.where(score is min).first['file'] basename
```

```
int lowestScore = min score
```

```
String highestFile = results.where(score is max).first['file'] basename
```

```
int highestScore = max score
```

```
6. If bonsai run fails: print 'SKIP : analysis failed', continue
```

```
After all repos: print markdown table:
```

```
| Repo | Files | Avg Score | Lowest File | Lowest | Highest File | Highest |  
|-----|-----|-----|-----|-----|-----|-----|  
| ... |
```

Also print a one-line summary:

```
'Across X repos and Y files, average bonsai score: Z/100'
```

IMPLEMENT final README.md:

```
# ■ bonsai
```

```
> Prune your widget trees before your users feel them.
```

```
[![CI](badge url)](link) [![pub.dev](badge url)](link)
```

bonsai is a static analysis tool for Flutter that measures widget tree health at the source-code level. It finds every reactive scope in your build() methods, counts the widgets inside each one, and turns that into a health score per file. No running the app. No build step. Works in CI.

```
## Install
```

```
dart pub global activate bonsai
```

```
## Usage
```

```
# Analyse a Flutter project
```

```
bonsai analyze lib/
```

```
# JSON output for custom reporters
```

```
bonsai analyze lib/ --format json
```

```
# CI gate: fail if any file scores below 60
```

```
bonsai analyze lib/ --threshold 60 --exit-code
```

```
## Example output
```

```
home_screen.dart 31/100 RED
```

```
-> Extract widgets inside reactive scopes – blast score is HIGH
```

```
profile_screen.dart 74/100 YELLOW
```

```
settings_screen.dart 91/100 GREEN
```

```
Analysed 3 files. 2 passed. 1 failed (below threshold 50).
```

```
## Metrics
```

**Blast Score** – widget count inside each reactive scope (setState, StreamBuilder, Consumer, etc). Directly predicts rebuild cost per state change.

**Composability Pressure** – ratio of extracted custom widgets to raw leaf widgets in build(). Measures decomposition quality.

**Context Coupling Depth** – nesting level of context.watch/read calls. High depth means unnecessary rebuilds cascade through the tree.

```
## VS Code
```

Install the bonsai VS Code extension. Health score appears as a gutter dot on your build() method, updated on every save.

```
## Contributing
```

See CONTRIBUTING.md

## License

MIT

IMPLEMENT CONTRIBUTING.md:

Brief guide on:

- How to add a new reactive scope to kReactiveScopes in scopes.dart
- How to add a new leaf widget to kLeafWidgets in composability.dart
- How to add a new metric: models.dart -> visitor.dart -> aggregator.dart -> tests
- PR checklist: dart analyze zero issues, dart test passes, coverage >= 85%

CONSTRAINTS:

- CI must use ubuntu-latest and dart stable channel – NO Flutter SDK in CI
- Publish workflow triggers ONLY on tags matching v\*.\*.\* – never on branch push
- Benchmark script must handle clone and analysis failures gracefully – skip and continue
- README example output must use actual expected scores from fixtures
- Do NOT hardcode pub.dev credentials – use PUB\_CREDENTIALS secret

VERIFY:

- Push to main branch – CI workflow must go green in GitHub Actions
- dart run scripts/benchmark.dart – must print markdown table (repos will clone slowly, ~5min)
- README renders correctly on GitHub

#### EXPECTED OUTPUT — WHAT DONE LOOKS LIKE

- ✓ CI workflow passes on main branch push
- ✓ publish.yml only triggers on v\*.\*.\* tags
- ✓ benchmark.dart prints markdown table without crashing
- ✓ README renders correctly on GitHub with correct example output

#### VERIFY BEFORE RUNNING NEXT PROMPT

*Push a commit to main. Go to GitHub Actions tab. CI workflow must pass green. Run dart run scripts/benchmark.dart locally (will take a few minutes to clone repos).*

**P8**

### Tag v0.1.0 + LinkedIn post + final polish

Day 6 | Est: 1 hr

Copy the prompt block below exactly into Codex / Claude Code

#### PROMPT — COPY THIS ENTIRE BLOCK

You are working on 'bonsai', a Flutter static analysis tool.

This is the final prompt – prompt 8 of 8.

The tool is built. This prompt handles final polish and launch.

YOUR JOB IN THIS PROMPT:

1. DART FORMAT – run on everything:

cd bonsai/packages/bonsai\_core && dart format .

cd bonsai/packages/bonsai\_cli && dart format .

Commit all formatting changes.

## 2. FINAL dart analyze:

Run dart analyze on both packages. Zero issues required.

If there are issues: fix them all before continuing.

## 3. UPDATE VERSION NUMBERS:

In bonsai\_core/pubspec.yaml: version: 0.1.0

In bonsai\_cli/pubspec.yaml: version: 0.1.0

In bonsai\_vscode/package.json: "version": "0.1.0"

## 4. ADD pub.dev METADATA to bonsai\_core/pubspec.yaml:

description: 'Static widget tree analyzer for Flutter. Measures blast radius, composability, and context coupling depth. Gives every screen a health score.'

homepage: [https://github.com/YOUR\\_USERNAME/bonsai](https://github.com/YOUR_USERNAME/bonsai)

repository: [https://github.com/YOUR\\_USERNAME/bonsai](https://github.com/YOUR_USERNAME/bonsai)

issue\_tracker: [https://github.com/YOUR\\_USERNAME/bonsai/issues](https://github.com/YOUR_USERNAME/bonsai/issues)

topics:

- flutter
- dart
- static-analysis
- widget
- performance

## 5. ADD EXAMPLE FILE for pub.dev:

Create bonsai\_core/example/main.dart:

```
import 'package:bonsai_core/bonsai_core.dart';

void main() {
  // Analyse a single Dart file
  final result = analyzeFile('path/to/your/screen.dart');
  print(result.health); // e.g. 74/100 [YELLOW]
  print('Blast Score: ${result.metrics.maxBlastScore}');
  print('Composability: ${result.metrics.composabilityRatio}');
  print('Context Depth: ${result.metrics.maxContextDepth}');
}
```

## 6. CREATE .bonsai.yaml EXAMPLE CONFIG:

Create bonsai/.bonsai.yaml with content:

```
# bonsai configuration
# Copy to your Flutter project root
threshold: 60 # files below this score fail CI
exclude_paths:
  - lib/generated/
  - lib/l10n/
# Future config options (not yet implemented):
# weights:
# blast: 0.4
```

```
# composability: 0.3
```

```
# context_depth: 0.3
```

#### 7. FINAL COMMIT:

```
git add -A
```

```
git commit -m 'chore: prepare v0.1.0 release'
```

#### 8. TAG AND PUSH:

```
git tag v0.1.0
```

```
git push origin main
```

```
git push origin v0.1.0
```

This triggers the publish.yml workflow on GitHub Actions.

#### 9. WRITE THE LINKEDIN POST (do not post yet – save as linkedin\_post.md in repo root):

```
---
```

LINKEDIN POST CONTENT TO WRITE IN linkedin\_post.md:

I built a tool I couldn't find anywhere.

Flutter developers know when a screen is messy. What we didn't have was a number that tells you *\*how bad it is\** – statically, without running the app.

Introducing bonsai – a static analyzer for Flutter that does one thing:

it finds every reactive scope in your widget tree (setState, StreamBuilder, Consumer, BlocBuilder), counts the widgets inside each one, and tells you the blast radius before your users feel it.

Here's what it looks like:

```
home_screen.dart 31/100 RED
```

```
-> Extract widgets inside reactive scopes
```

```
profile_screen.dart 74/100 YELLOW
```

```
settings_screen.dart 91/100 GREEN
```

Three metrics under the hood:

- Blast Score: widget count inside reactive scopes (the rebuild bomb)
- Composability Pressure: ratio of leaf widgets to extracted classes
- Context Coupling Depth: how deep context.watch is called

Works in CI. Works in VS Code. Pure static analysis – no running the app.

I ran it against 10 open-source Flutter repos. Results in the benchmark table in the README.

Open source. Dart pub.dev. VS Code marketplace.

[github.com/YOUR\\_USERNAME/bonsai](https://github.com/YOUR_USERNAME/bonsai)

If you work with Flutter, star it. If you know someone who does, tag them.

#Flutter #Dart #OpenSource #MobileEngineering #SoftwareEngineering

```
---
```

#### 10. VERIFY EVERYTHING:

```
dart pub publish --dry-run in bonsai_core/ (should pass all checks)
```

```
dart pub publish --dry-run in bonsai_cli/ (should pass all checks)
```

Check GitHub Actions — publish workflow triggered by v0.1.0 tag

Check pub.dev after ~10 min — packages should appear

#### CONSTRAINTS:

- Do NOT publish until dry-run passes with zero issues
- Replace YOUR\_USERNAME with your actual GitHub username everywhere
- The LinkedIn post in linkedin\_post.md must be exactly as written above

with your real benchmark numbers filled in from benchmark.dart output

- Do not skip the dart format step — pub.dev will reject unformatted code

#### EXPECTED OUTPUT — WHAT DONE LOOKS LIKE

- ✓ dart format shows no changes (all already formatted)
- ✓ dart analyze shows zero issues on both packages
- ✓ dart pub publish --dry-run passes in both packages
- ✓ v0.1.0 tag triggers publish workflow in GitHub Actions
- ✓ linkedin\_post.md exists in repo root with benchmark numbers filled in

#### VERIFY BEFORE RUNNING NEXT PROMPT

*Run: dart pub publish --dry-run in both packages. Must pass. Push v0.1.0 tag. Go to GitHub Actions — publish workflow must trigger. Wait 10 min then check pub.dev for bonsai\_core and bonsai\_cli.*

## 5. Post-Launch — What to Do With It

#### THE POST THAT GETS SHARED

Do not post until you have run the benchmark and have real numbers from real repos. Numbers about real projects are what gets shared. 'I analysed flutter/gallery and found 4 screens with blast scores above 60' is a story. 'I built a static analyzer' is not.

#### COMMUNITIES — IN THIS ORDER

- LinkedIn — post linkedin\_post.md. Tag 3 Flutter engineers you know personally.
- r/FlutterDev — 'I built a static widget tree analyzer, here are benchmark results on 10 popular repos'
- Flutter Discord #tools channel — drop the GitHub link with one paragraph
- pub.dev listing — detailed description, example output, the benchmark table in the README
- Hacker News Show HN — only after benchmark post with real numbers

#### THE METRIC THAT MATTERS FOR YOUR GITHUB PROFILE

Stars come from the benchmark post, not the launch post. Run bonsai against the 10 biggest Flutter OSS repos. The more surprising the numbers, the more the post spreads. If you find that a well-known app has a screen with blast score 90+, that is the headline.

#### AFTER V0.1.0 — WHAT MAKES THIS A RESEARCH CONTRIBUTION

- Widget clone detection — structural AST diff across files. Finds copy-pasted widget subtrees. No existing tool does this. This is publishable.
- Trigger classification — distinguish setState in scroll listener (bad) from setState in button tap (fine). Same blast score but very different actual risk.

- Historical scoring — track health score per file over git commits. Show the trend as you refactor.

Any one of these three, written up and posted on arXiv as a short paper, gives you a credible research contribution as a 3rd-year SE student. The tool is the implementation. The paper is the writeup.

---

■ **bonsai — prune your widget trees before your users feel them.**

[github.com/YOUR\\_USERNAME/bonsai](https://github.com/YOUR_USERNAME/bonsai)